

Mobile App Capability Catalog & Geminio OS Mapping

This document outlines the complete mobile app development capabilities as specified in the `mobile-app-capabilities.pdf` document and maps them against the current implementation status within the Geminio OS project.

1. UI / UX Design

Feature	Description	Geminio OS Status	Notes
Expo Router (file-based nav)	Pages are files — create a file, get a route. Handles stacks, tabs, drawers, modals, and deep links automatically	✅ Implemented	Core navigation is based on Expo Router.
Shared Element Transitions	Animate a card expanding into a detail screen — the element visually "flies" between screens	❌ Missing	Requires implementation using <code>react-native-reanimated</code> for smooth transitions.
Gesture Handler	Swipe-to-delete, drag-to-reorder, pinch-to-zoom — all running on the native thread (no JS jank)	❌ Missing	Requires <code>react-native-gesture-handler</code> integration for advanced interactions.
Reanimated 3	60fps animations driven by native worklets — spring physics, interpolations, scroll-linked effects	✅ Implemented	<code>react-native-reanimated</code> is already a dependency.
Safe Area Insets	Automatically respects notches, Dynamic Island, home indicators, and status bars on every device	✅ Implemented	<code>ScreenContainer</code> component handles this.
Keyboard Avoidance	Form fields slide up when the keyboard	❌ Missing	Requires <code>KeyboardAvoidingView</code> or similar logic.

	appears so nothing is hidden behind it		
Haptic Feedback	Trigger device vibration patterns (light, medium, heavy, selection, success, warning, error)	✅ Implemented	<code>expo-haptics</code> is used in <code>haptic-tab.tsx</code> .
Skeleton Loaders	Show animated placeholder shapes while data loads — better UX than spinners	❌ Missing	Custom components needed for data loading states.
Bottom Sheet	Draggable panel that slides up from the bottom — used for menus, pickers, detail views	❌ Missing	Requires a third-party library or custom implementation.
Flash List	Ultra-fast replacement for <code>FlatList</code> — handles 10,000+ items without frame drops	❌ Missing	Currently using <code>FlatList</code> or <code>ScrollView</code> . <code>FlashList</code> needs to be integrated.
Masonry Layout	Pinterest-style variable-height grid layouts	❌ Missing	Custom component or library needed.
Parallax Scroll	Background images scroll slower than foreground content for depth	❌ Missing	Custom implementation using <code>react-native-reanimated</code> .
Dark / Light Mode	Automatic theme switching based on system preference, with manual override	✅ Implemented	Handled by <code>useColorScheme</code> and <code>ThemeProvider</code> .
Custom Fonts	Load any Google Font or local typeface via <code>expo-font</code>	❌ Missing	<code>expo-font</code> is a dependency, but no custom fonts are loaded.
Blur Views	Native blur effect behind modals and overlays (iOS frosted glass, Android blur)	❌ Missing	Requires <code>expo-blur</code> or similar.

Linear Gradient	Smooth color gradients on any view or background	✗ Missing	Requires <code>expo-linear-gradient</code> or similar.
SVG Support	Render fully scalable vector graphics via <code>react-native-svg</code>	✓ Implemented	<code>react-native-svg</code> is a dependency.
Lottie Animations	Play complex After Effects animations as lightweight JSON files	✗ Missing	Requires <code>lottie-react-native</code> .
Confetti / Particle Effects	Celebration animations for success states	✗ Missing	Custom implementation or library.
Design Exploration on Canvas	Prototype multiple UI variants side-by-side visually before writing any code	✗ Missing	Requires integration with a design tool or custom prototyping feature.

2. Navigation & Routing

Feature	Description	Geminio OS Status	Notes
Stack Navigation	Push/pop screens with native slide transitions	✓ Implemented	Handled by Expo Router.
Tab Navigation	Bottom tab bar with icons and badges	✓ Implemented	Handled by Expo Router <code>(tabs)/_layout.tsx</code> .
Drawer Navigation	Side menu that slides in from the left or right	✗ Missing	Requires <code>react-navigation/drawer</code> .
Modal Screens	Full-screen overlays with their own navigation stack	✓ Implemented	Handled by Expo Router.
Deep Linking	External URLs open directly to a specific screen inside the app	✓ Implemented	Configured in <code>app.config.ts</code> .

Universal Links	Web URLs (https://) open the app instead of the browser if installed	✅ Implemented	Configured in <code>app.config.ts</code> .
Dynamic Routes	Route parameters like <code>/user/[id]</code> — works like Next.js file-based routing	✅ Implemented	Handled by Expo Router.
Auth Guard Routes	Redirect unauthenticated users to login automatically	✅ Implemented	Handled by <code>use-auth.ts</code> and <code>_layout.tsx</code> .
Tab Badges	Show notification counts on tab bar icons	❌ Missing	Requires custom implementation for badges.
Back Button Handling	Custom behavior when Android back button is pressed	❌ Missing	Requires <code>useBackHandler</code> hook or similar.

3. App Logic & State

Feature	Description	Geminio OS Status	Notes
React Query (TanStack)	Fetches, caches, deduplicates, and background-refreshes server data automatically	✅ Implemented	<code>trpc.ts</code> uses TanStack Query.
Optimistic Updates	UI updates instantly before the server confirms — rolls back on error	❌ Missing	Requires explicit implementation with React Query mutations.
Infinite Scroll / Pagination	Load more items as the user scrolls — built into React Query	❌ Missing	Requires specific implementation with React Query.
Offline Detection	Detect when the device loses connection and queue requests for retry	❌ Missing	Requires <code>NetInfo</code> or similar.

Background Fetch	Run code periodically in the background even when the app is not open	✗ Missing	Requires <code>expo-background-fetch</code> or <code>expo-task-manager</code> .
App State Tracking	Detect when the app goes to background/foreground — refresh data on return	✗ Missing	Requires <code>AppState</code> API.
Local Notifications (Scheduled)	Schedule notifications to fire at a specific time locally, no server needed	✗ Missing	Requires <code>expo-notifications</code> .
Remote Push Notifications	Receive server-triggered push notifications via Expo Push API	✓ Implemented	<code>expo-notifications</code> is a dependency, and backend has notification capabilities.
In-App Notifications	Toast/snackbar messages inside the app for success, error, and info states	✗ Missing	Requires custom UI components.
Focus/Blur Hooks	Run code when a screen gains or loses focus (e.g., refresh data when navigating back)	✗ Missing	Requires <code>useFocusEffect</code> from <code>react-navigation</code> .
Clipboard	Read from and write to the device clipboard	✗ Missing	Requires <code>expo-clipboard</code> .
Share Sheet	Trigger the native OS share dialog to share text, URLs, or files	✗ Missing	Requires <code>expo-sharing</code> .
Vibration Patterns	Custom vibration sequences for alerts and feedback	✓ Implemented	<code>expo-haptics</code> is used.

4. Backend & API

Feature	Description	Geminio OS Status	Notes
Express 5 API Server	Shared backend in the monorepo — add routes, middleware, and handlers	✅ Implemented	<code>server/_core/index.ts</code> is an Express server.
OpenAPI Spec (Contract-First)	Define every endpoint in YAML before writing code — the single source of truth	❌ Missing	No OpenAPI spec found.
Auto-Generated React Query Hooks	Codegen produces typed hooks from the spec — no hand-written fetch calls	✅ Implemented	tRPC client generates hooks.
Zod Request Validation	Every incoming request is validated against a schema — invalid input is rejected immediately	✅ Implemented	Used in tRPC routers.
Zod Response Validation	Validate what the server sends back — catch server bugs at the boundary	❌ Missing	Needs explicit implementation in tRPC.
Rate Limiting	Limit how many requests a client can make per minute to prevent abuse	❌ Missing	Requires middleware implementation.
Request Logging	Every API request is logged with method, path, status, and duration	✅ Implemented	Basic logging is present in Express.
Error Handling Middleware	Catch all unhandled errors and return consistent JSON error responses	✅ Implemented	Present in Express server.
File Upload Handling	Accept images and files from the device with multipart form processing	❌ Missing	Requires <code>multer</code> or similar.
Streaming Responses	Stream large responses chunk by	❌ Missing	Requires specific Express

	chunk instead of waiting for the full response		implementation.
Health Check Endpoint	/api/healthz — confirms the backend is alive, used by monitoring and deployment	✅ Implemented	/api/healthz endpoint exists.
Webhook Receivers	Accept incoming webhooks from Stripe, GitHub, and other services	❌ Missing	Requires specific endpoint implementations.
CORS Configuration	Control which origins can call the API	✅ Implemented	Configured in Express.
Compression	Gzip API responses to reduce data transfer on mobile networks	❌ Missing	Requires compression middleware.

5. Data Storage

Feature	Description	Geminio OS Status	Notes
PostgreSQL (Drizzle ORM)	Full relational database with typed schema, migrations, and type-safe queries	✅ Implemented	Drizzle ORM with MySQL is used.
Schema Design & Migrations	Define tables, columns, relationships, indexes, and enums — push changes with one command	✅ Implemented	Drizzle Kit is used for migrations.
Soft Deletes	Mark records as deleted without removing them — enables undo and audit trails	❌ Missing	Requires schema modification and ORM logic.
Timestamps (auto)	createdAt and updatedAt populate	✅ Implemented	Handled by Drizzle schema.

	automatically on every write		
Full-Text Search	Search across text columns using PostgreSQL built-in FTS or pg_trgm	✗ Missing	Requires specific database configuration and query implementation.
AsyncStorage	Lightweight key-value store on the device — no backend needed	✓ Implemented	<code>AsyncStorage</code> is a dependency.
Expo SQLite	Full on-device relational database — works completely offline	✗ Missing	Not currently used.
Object Storage (App Storage)	Store and serve images, videos, PDFs, and other files — returns signed URLs	✓ Implemented	<code>server/storage.ts</code> handles S3-compatible storage.
Secure Store	Encrypted key-value storage on the device for tokens and sensitive values (Keychain / Keystore)	✓ Implemented	<code>expo-secure-store</code> is a dependency.
MMKV	Ultra-fast synchronous key-value storage — much faster than AsyncStorage for frequent reads	✗ Missing	Requires <code>react-native-mmkv</code> .

6. Authentication & Authorization

Feature	Description	Geminio OS Status	Notes
Clerk Auth	Email/password, social login (Google, Apple, GitHub), magic links, MFA — fully managed	✗ Missing	Currently using custom auth.

Replit Auth	"Sign in with Replit" via OpenID Connect — zero config	❌ Missing	Not applicable in this context.
Biometric Auth	Face ID and fingerprint unlock via expo-local-authentication	✅ Implemented	expo-local-authentication is a dependency.
Session Management	Server-side sessions persisted to the database with secure cookie handling	✅ Implemented	Handled by server/_core/auth.ts .
JWT Tokens	Stateless token-based auth for APIs that need it	✅ Implemented	Used for API authentication.
Role-Based Access Control (RBAC)	Admin, user, guest roles — enforced at the API and UI levels	✅ Implemented	Basic RBAC is present.
Auth Guards	Redirect unauthenticated users automatically — no manual checks per screen	✅ Implemented	Handled by _layout.tsx and use-auth.ts .
Refresh Token Rotation	Silently refresh expired access tokens without logging the user out	❌ Missing	Requires explicit implementation.
Account Linking	Connect multiple login providers to a single account	❌ Missing	Requires custom implementation.
User Profile Management	Update display name, avatar, email, and password through prebuilt Clerk UI	✅ Implemented	Basic user profile management is present.

7. AI / Machine Learning

Feature	Description	Geminio OS Status	Notes
OpenAI (GPT-4o, o1)	Chat, summarization, classification,	✅ Implemented	server/_core/llm.ts integrates with

	extraction, code generation — Replit-proxied, no key needed		OpenAI.
Anthropic (Claude)	Long-context reasoning, document analysis, structured output — Replit-proxied	✗ Missing	Requires integration with Anthropic API.
Gemini	Multimodal tasks (text + image understanding) — Replit-proxied	✓ Implemented	<code>server/_core/llm.ts</code> integrates with Gemini.
OpenRouter	Access 100+ models (Llama, Mistral, Cohere, Qwen) through one API — Replit-proxied	✗ Missing	Requires integration with OpenRouter API.
Streaming AI Output	Token-by-token streaming displayed in real time — chat-like feel	✗ Missing	Requires frontend and backend streaming implementation.
Structured JSON Output	Force AI to respond in a validated JSON schema — no parsing guesswork	✓ Implemented	Used in AI Builder modules.
Function Calling / Tool Use	AI decides when to call your app's functions (search DB, send email, fetch data)	✓ Implemented	Used in AI Agent Builder.
Text Embeddings	Convert text to vectors for semantic search, similarity matching, recommendations	✗ Missing	Requires integration with embedding models.
Image Analysis	Send device photos to a vision model for description, OCR, or classification	✓ Implemented	<code>server/_core/imageGeneration.ts</code> has image analysis capabilities.
Voice Transcription (Whisper)	Transcribe audio recordings to text	✓ Implemented	<code>server/_core/voiceTranscription.ts</code> uses Whisper.

Text-to-Speech	Convert text to natural-sounding audio via OpenAI TTS	❌ Missing	Requires integration with TTS API.
AI Image Generation	Generate images from prompts via DALL-E or Stable Diffusion	✅ Implemented	<code>server/_core/imageGeneration.ts</code> handles image generation.
On-Device ML (TensorFlow Lite)	Run lightweight ML models locally without a network call	❌ Missing	Requires <code>expo-tfjs</code> or similar.

8. Device Hardware & Native APIs

Feature	Description	Geminio OS Status	Notes
Camera	Capture photos and video with full control over flash, zoom, and facing	❌ Missing	Requires <code>expo-camera</code> .
Photo Library	Pick images and videos from the device gallery	❌ Missing	Requires <code>expo-image-picker</code> .
Location (GPS)	Get current coordinates, watch position over time, and reverse-geocode addresses	❌ Missing	Requires <code>expo-location</code> .
Background Location	Track location even when the app is in the background	❌ Missing	Requires <code>expo-location</code> and background tasks.
Accelerometer / Gyroscope	Read raw sensor data for motion-based features	❌ Missing	Requires <code>expo-sensors</code> .
Barcode / QR Scanner	Scan barcodes and QR codes with the camera	❌ Missing	Requires <code>expo-barcode-scanner</code> .
NFC	Read NFC tags on supported Android devices	❌ Missing	Requires <code>expo-nfc</code> .

Contacts	Read the device address book (with permission)	✗ Missing	Requires <code>expo-contacts</code> .
Calendar	Create and read calendar events	✗ Missing	Requires <code>expo-calendar</code> .
Microphone / Audio Recording	Record audio from the device microphone	✓ Implemented	<code>expo-audio</code> is a dependency.
Audio Playback	Play local and remote audio files	✓ Implemented	<code>expo-audio</code> is a dependency.
Video Playback	Play local and remote video with playback controls	✓ Implemented	<code>expo-video</code> is a dependency.
Network Info	Detect connection type (WiFi, cellular, offline) and bandwidth	✗ Missing	Requires <code>expo-network</code> .
Battery Status	Read battery level and charging state	✗ Missing	Requires <code>expo-battery</code> .
Brightness	Read and set screen brightness	✗ Missing	Requires <code>expo-brightness</code> .
Flashlight	Toggle the device torch	✗ Missing	Requires <code>expo-camera</code> (for torch).

9. Payments & Monetization

Feature	Description	Geminio OS Status	Notes
RevenueCat	In-app purchases and subscriptions using native iOS/Android billing — no Stripe needed	✗ Missing	Requires <code>react-native-purchases</code> .
Stripe (Web-backed)	Card payments, one-time charges, and subscriptions through the backend	✗ Missing	Requires Stripe integration.
Paywalls	Gate features behind a subscription or	✗ Missing	Requires RevenueCat or custom

	purchase — built-in paywall UI from RevenueCat		implementation.
Free Trial Management	Offer trials that automatically convert to paid after a period	✗ Missing	Requires RevenueCat or custom implementation.
Entitlement Checking	Know in real time whether a user has access to a feature	✗ Missing	Requires RevenueCat or custom implementation.
Receipt Validation	Validate Apple and Google purchase receipts server-side to prevent fraud	✗ Missing	Requires server-side implementation.
Promotional Offers	Discounted pricing for win-back and retention campaigns	✗ Missing	Requires RevenueCat or custom implementation.

10. Third-Party Integrations

Feature	Description	Geminio OS Status	Notes
Google Sheets	Read and write spreadsheet data — useful for simple backends or data exports	✗ Missing	Requires Google Sheets API integration.
Linear	Create and update issues from inside the app	✗ Missing	Requires Linear API integration.
GitHub	Read repos, issues, PRs — useful for developer tools	✗ Missing	Requires GitHub API integration.
Slack	Send messages or receive events	✗ Missing	Requires Slack API integration.
Notion	Read and write Notion database entries	✗ Missing	Requires Notion API integration.
HubSpot	Create contacts, deals, and activities	✗ Missing	Requires HubSpot API integration.

Jira	Create and update tickets	✗ Missing	Requires Jira API integration.
Snowflake / BigQuery / Databricks	Query data warehouses from the backend	✗ Missing	Requires data warehouse integration.
Mapbox / Google Maps	Interactive maps, directions, and location search	✗ Missing	Requires Mapbox/Google Maps SDK.
Twilio	Send SMS messages	✗ Missing	Requires Twilio API integration.
SendGrid / Resend	Send transactional emails	✗ Missing	Requires email API integration.
Expo Push API	Send push notifications to any device without managing APNs/FCM directly	✓ Implemented	<code>expo-notifications</code> is a dependency.

11. Testing & Quality

Feature	Description	Geminio OS Status	Notes
TypeScript End-to-End Type Safety	DB schema to API to mobile client — type errors caught at compile time, not runtime	✓ Implemented	Achieved with Drizzle ORM and tRPC.
LSP Diagnostics	Inspect every static error across the whole codebase at once	✓ Implemented	Supported by VS Code and TypeScript.
Zod Schema Validation	Runtime validation at every API boundary — bad data is caught before it causes damage	✓ Implemented	Used in tRPC routers.
Screenshot Verification	Take a screenshot of the running app to visually verify layout	✓ Implemented	<code>webdev_take_screenshot</code> tool used.

Log Inspection	Read workflow, server, and device console logs in real time	✅ Implemented	<code>tail -f</code> and <code>grep</code> used.
Error Boundaries	Catch component crashes and show a recovery UI instead of a blank screen	❌ Missing	Requires React Error Boundaries implementation.
Drizzle TypeScript Queries	Every DB query is type-checked — wrong column name or type is a compile error	✅ Implemented	Drizzle ORM provides this.
API Health Checks	Verify the backend is alive and responding before diagnosing frontend issues	✅ Implemented	<code>/api/healthz</code> endpoint exists.
Dependency Audit	Scan all packages for known CVEs and flag critical vulnerabilities	❌ Missing	Requires <code>npm audit</code> or similar.
SAST Scan	Static analysis to catch injection, secret leakage, and insecure patterns in source code	❌ Missing	Requires SAST tool integration.

12. Deployment & Distribution

Feature	Description	Geminio OS Status	Notes
EAS Build (Expo Application Services)	Build native iOS/Android apps in the cloud without Xcode/Android Studio	❌ Missing	Requires <code>eas build</code> configuration.
Over-the-Air (OTA) Updates	Push code updates instantly to users without app store review	❌ Missing	Requires EAS Update configuration.
App Store Connect Integration	Automate submitting iOS builds to TestFlight and App Store	❌ Missing	Requires <code>fastlane</code> or custom scripts.

Google Play Console Integration	Automate submitting Android builds to internal/alpha/beta tracks	✗ Missing	Requires <code>fastlane</code> or custom scripts.
Web Deployment (Vercel/Cloudflare)	Deploy web build to CDN for fast global access	✓ Implemented	Web build is static and can be deployed to Vercel/Cloudflare.
Desktop Bundling (Electron/Tauri)	Package web app as native desktop app for Windows, macOS, Linux	✗ Missing	Requires Electron/Tauri integration.
CI/CD Pipelines	Automate testing, building, and deployment on every code push	✗ Missing	Requires GitHub Actions or similar.
Release Management	Manage versions, changelogs, and rollbacks	✗ Missing	Requires custom tooling or platform features.

13. Performance & Optimization

Feature	Description	Geminio OS Status	Notes
Bundle Size Optimization	Reduce app download size by removing unused code and assets	✗ Missing	Requires <code>metro-bundle-analyzer</code> and manual optimization.
Image Optimization	Compress and resize images for faster loading	✓ Implemented	<code>expo-image</code> handles some optimization.
Font Preloading	Load fonts before they are needed to prevent text reflow	✗ Missing	Requires <code>expo-font</code> preloading.
Code Splitting (Dynamic Imports)	Load only necessary code for each screen/feature	✗ Missing	Requires custom implementation with React Native.
Caching Strategies	Cache API responses, images, and other	✓ Implemented	React Query handles API caching.

	assets to reduce network requests		
Fast Refresh (HMR)	Instant feedback during development without losing app state	✅ Implemented	Default Expo feature.
Native Module Optimization	Ensure native modules are performant and don't block UI thread	✅ Implemented	<code>react-native-reanimated</code> uses native worklets.
Performance Monitoring	Track app performance in production (FPS, memory, network)	❌ Missing	Requires <code>Sentry</code> or similar.
Background Threading (Web Workers)	Offload heavy computations to background threads to keep UI responsive	❌ Missing	Requires <code>react-native-web-workers</code> or similar.

14. Security

Feature	Description	Geminio OS Status	Notes
Data Encryption (at rest)	Encrypt sensitive data stored on device and in database	✅ Implemented	<code>expo-secure-store</code> and database encryption.
Data Encryption (in transit)	Encrypt all network communication with HTTPS/TLS	✅ Implemented	Default for HTTPS.
Input Validation (frontend)	Validate user input before sending to backend	✅ Implemented	Basic frontend validation.
Input Validation (backend)	Validate all incoming API requests to prevent malicious data	✅ Implemented	Zod validation in tRPC.
Secure Coding Practices	Follow OWASP guidelines to prevent common vulnerabilities	✅ Implemented	General good practices.

API Key Management	Securely store and access API keys	✅ Implemented	Handled by environment variables.
Vulnerability Scanning	Regularly scan codebase and dependencies for known vulnerabilities	❌ Missing	Requires <code>npm audit</code> or similar.
Penetration Testing	Simulate attacks to find security weaknesses	❌ Missing	Requires external security audit.
MFA (Multi-Factor Authentication)	Add an extra layer of security for user logins	❌ Missing	Requires custom implementation or Clerk Auth.
Session Hijacking Prevention	Protect against unauthorized access to user sessions	✅ Implemented	Secure cookie handling.
Rate Limiting	Prevent brute-force attacks and API abuse	❌ Missing	Requires middleware implementation.
Content Security Policy (CSP)	Mitigate XSS attacks by restricting resource loading	❌ Missing	Requires web configuration.

15. Developer Experience & Hidden Capabilities

Feature	Description	Geminio OS Status	Notes
Hot Module Replacement (HMR)	Instant updates to code without full app reload	✅ Implemented	Default Expo feature.
Debugging Tools (Flipper/React DevTools)	Inspect UI, network, and performance	✅ Implemented	Supported by Expo.
Type Checking (TypeScript)	Catch errors at compile time	✅ Implemented	TypeScript is used.
Linting (ESLint/Prettier)	Enforce code style and best practices	✅ Implemented	ESLint and Prettier are configured.
Automated Testing (Jest/Vitest)	Run unit, integration, and end-to-end tests	✅ Implemented	Vitest is used.

Code Generation (CLI)	Generate boilerplate code for components, screens, APIs	❌ Missing	Requires custom CLI or scaffolding.
Storybook / Style Guide	Document and test UI components in isolation	❌ Missing	Requires Storybook integration.
Environment Variable Management	Securely manage different configurations for dev/prod	✅ Implemented	Handled by <code>dotenv</code> and <code>app.config.ts</code> .
Local Development Server	Run backend and frontend locally	✅ Implemented	<code>pnpm dev</code> script.
Offline Development	Work on the app without an internet connection	✅ Implemented	Most features can be developed offline.

16. Maintenance & Long-Term

Feature	Description	Geminio OS Status	Notes
Codebase Documentation	Comprehensive documentation for all modules and APIs	✅ Implemented	<code>PROJECT_SUMMARY.md</code> , <code>API_DOCUMENTATION.md</code> , <code>DEPLOYMENT_GUIDE.md</code> , inline comments.
Version Control (Git)	Track changes and collaborate	✅ Implemented	Git is used.
Dependency Management (pnpm)	Manage project dependencies efficiently	✅ Implemented	pnpm is used.
Regular Updates (Expo SDK)	Keep up-to-date with latest features and security patches	✅ Implemented	Expo SDK 54 is used.
Scalable Architecture	Design for future growth and increased user load	✅ Implemented	Modular architecture.

Monitoring & Alerting	Track application health and notify on issues	✗ Missing	Requires Sentry , Datadog , or similar.
Logging & Tracing	Centralized logging and distributed tracing for debugging	✗ Missing	Requires Winston , Pino , or similar.
Backup & Restore	Database and file storage backup strategy	✓ Implemented	Database migrations and S3 storage.
Disaster Recovery	Plan for recovering from major outages	✗ Missing	Requires comprehensive DR plan.
Long-Term Support (LTS)	Plan for maintaining the app over many years	✓ Implemented	Standard practices.